

THE FORGOTTEN ATTACK SURFACE

How One Forgotten Sign-Out Empties a Lab — and How NSU Can Close the Hole for Free

Prepared for: the senior NSU leader who gave Sajid the opening **Author:** Sajid (NSU CSE, final year) **Frame:** Don't put juicy food where users can knock it over. Take the food off the table.

0. ONE-MINUTE PITCH (read this in the elevator)

A student leaves a lab PC without signing out of Chrome. The next student opens `chrome://settings/passwords`, clicks the eye icon next to any saved password, types the local Windows password (often weak, often shared), and **walks away with every saved credential** of the previous user — their @northsouth.edu mail, Facebook, bKash, bank, everything. They can export the entire vault as CSV in two clicks. They never installed malware. They never ran an exploit. They used the browser exactly the way Chrome is designed to work.

Worse: if **Chrome Sync is enabled** (the default invitation, the "turn on sync to access your bookmarks everywhere" banner that students click without reading), that same lab PC has now downloaded *everything* from that student's personal phone — saved Wi-Fi passwords, autofill addresses, payment methods, browsing history, even open tabs from their bedroom — into the lab profile. Even after the student signs out, **the local profile data persists on disk** in `%LOCALAPPDATA%\Google\Chrome\User Data\Profile X\` until somebody wipes it.

This is the **forgotten attack surface**. It is not phishing. It is not malware. It is the *default browser behaviour* on shared university endpoints.

The fix is free. Two Chrome enterprise policies (`SyncDisabled=1`, `BrowserSignin=0`), Windows Shared PC Mode (built into Windows 10/11, zero license cost), and a handful of Microsoft Defender ASR rules already on every NSU machine — and the entire class of attack is gone. No new software. No procurement cycle. No training campaign.

Ask: 30-day permission to deploy these on one lab as a pilot. Measurable baseline before/after. Then a paper.

1. THE ATTACK TREE — what actually happens on NSU lab machines today

Three classes. Class A and Class B are happening **right now** on NSU lab PCs without anyone doing anything malicious. Class C is what happens when one bad actor decides to weaponize it. Class D, E, F are added attacker classes.

Class A — "Forgotten Profile" (zero skill, zero tools, 30 seconds)

1. Student A signs into Chrome on a lab PC. Clicks "Turn on sync." Closes Chrome.
2. Student A walks away thinking signing out of Gmail is enough. **It isn't.**
3. Student B sits down. Opens Chrome. The previous profile is still there in the top-right avatar selector. Even after a sign-out, the local profile folder remains.
4. Student B opens `chrome://settings/passwords`. Every saved password is listed. Clicks the eye icon → prompts for the **Windows password** of the current logged-in account (the lab account, often shared, often `student123` or similar).
5. CSV export. Done.

What's leaked: every site password Student A ever saved to Chrome (Gmail, Facebook, Instagram, bKash, banking, university SIS, anything).

Class B — "Sync Bleed" (zero skill, victim does the work)

1. Student A uses Chrome on personal phone. Sync is on by default for everything: passwords, autofill, history, open tabs, payment methods, addresses, even browser extensions.
2. Student A signs into Chrome on a lab PC, "to check email quickly," opts to turn on sync because Chrome asks helpfully.
3. **Chrome downloads the entire personal sync set to the lab PC's local profile folder.** Saved Wi-Fi credentials from home. Credit card autofill. Address. Open tabs from last night. All of it. Now stored on disk at `%LOCALAPPDATA%\Google\Chrome\User Data\`.
4. Student A signs out and leaves. The local cached copy of the synced data **persists** until something deletes it.
5. Anyone who runs a local extractor (free tool, see Class C) gets the entire synced vault — not just what was typed on the lab PC, but everything that lived on the personal device.

This is the part most people don't understand. Sync is not just "upload my bookmarks." Sync is "make this device a full mirror of my private life."

Class C — "Local Extraction" (5 minutes, free tools, no malware install needed if attacker has shell access)

Once you have the Chrome `Login Data` SQLite database and the local DPAPI context (i.e., the same Windows user session), decryption is a one-liner.

Free, public, well-documented tools that read browser saved passwords offline:

- **ChromePass** (NirSoft) — single executable, point-and-click, reads `Login Data`, decrypts via DPAPI under the current user, exports CSV. No installation. No admin needed.
- **LaZagne** (open source, github.com/AlessandroZ/LaZagne) — single Python script or compiled binary, walks every browser (Chrome, Edge, Firefox, Opera, Brave), every

password store, every Wi-Fi profile, every mail client. One command, full dump.

- **Mimikatz** (`dpapi::chrome` module) — more capable, used by professionals; not needed for a casual attacker.
- **HackBrowserData** (github.com/moonD4rk/HackBrowserData) — newer, faster, supports App-Bound Encryption bypass on Chrome 127+.

All of these work as a **standard user** (no admin needed) against the **current user's** browser data. On a shared lab PC with a shared local account, "current user" means any student who sits down.

The Login Data is in: `C:\Users\<user>\AppData\Local\Google\Chrome\User Data\Default\Login Data` (SQLite) Encrypted blobs in the `password_value` column, decryptable by anyone with the same Windows user context via the DPAPI `CryptUnprotectData` Win32 call. That's it. That's the entire "security."

Class D — "Malicious Extension" (incognito doesn't save you)

Even if a student is careful, never saves passwords in Chrome, always uses incognito — a **malicious browser extension reads every keystroke on every page**, regardless of incognito.

- **Cyberhaven supply-chain attack (Dec 24, 2024)**: a phishing email to the extension developer let attackers push a malicious update to ~**400,000 users**, exfiltrating OAuth tokens, session cookies, and credentials from Google Workspace, Slack, Jira. Detection took **days**.
- **Carnegie Mellon ISO confirmed 20+ Chrome extensions** were compromised in the same wave by phishing their developers' Web Store credentials — students who installed these months earlier got auto-updated to the malicious version with no warning.
- More recent campaigns (Phantom Shuttle credential-stealing proxy extensions, December 2025; ShotBird / QuickLens ownership-transfer attacks, late 2025–early 2026) show **3.2 million+ users** exposed to extensions doing keystroke logging and stealthy exfiltration *after* a benign extension was sold or hijacked. Some had "Featured" badges in the Chrome Web Store.

Manifest V3 does not save you. It restricts *some* network APIs, but extensions can still read every form field, every keystroke, every page DOM via content scripts with `<all_urls>` permission. The Tackker-class extensions you mentioned — they exist, they're real, they pass review, and they live in Chrome Web Store for months before takedown.

Class E — "Native Hidden Keylogger" (Python, cmd, PowerShell — no install required on most lab PCs)

A first-year CSE student can write this in 20 lines. I'm not putting working code here, but the architecture is:

- Python `pynput` or `keyboard` library: hooks the Windows keyboard input. Captures every keystroke.
- Run with `pythonw.exe` (the windowless Python interpreter — **no console window appears**).
- Persistence via `HKCU\Software\Microsoft\Windows\CurrentVersion\Run` registry key, or a scheduled task created with `schtasks /create`, or a `.lnk` in the Startup folder. No admin needed for HKCU or per-user scheduled tasks.
- Exfiltration via Telegram Bot API, Discord webhook, or SMTP to a throwaway Gmail. Lab PCs at NSU almost certainly do not block outbound HTTPS to `api.telegram.org` or `discord.com`.

What makes lab PCs *paradise* for this:

- Students often have **local admin** or, even when they don't, `%LOCALAPPDATA%` is writable.
- Python is pre-installed on CSE lab machines.
- Defender's ASR rules are off by default.
- No application whitelisting (AppLocker / WDAC).
- No PowerShell Constrained Language Mode.
- No outbound network policy that would block Telegram/Discord.

A student installs this on one lab PC Monday morning. By Friday they have every keystroke of every student who used that machine: NSU credentials, Facebook chats, bKash PINs, exam answers being typed into Google Docs.

Class F — "Hardware Keylogger" (mentioning for completeness)

A KeyGrabber USB inline keylogger costs ~**USD 50** on AliExpress. Plug it between the keyboard and PC. Come back the next day, plug it into your laptop, retrieve a text file of everything typed.

Defends against: no software-only solution. Only physical inspection of every USB port + tamper-evident lab inspections work. Mention it in the paper; don't make it the focus.

2. WHY THIS HASN'T BEEN ADDRESSED ANYWHERE (your thesis hook)

This is the angle that makes the paper publishable: **the entire industry has been focused on phishing simulations and "user training,"** while the architectural attack surface on shared endpoints sits wide open at almost every university.

The **2024 Snowflake breach** is the canonical example. Mandiant found that **79.7% of compromised accounts had prior credential exposure via infostealers** (Vidar, RisePro, RedLine, Raccoon, Lumma, MetaStealer). The credentials weren't phished. They were lifted from browser local databases — exactly the Chrome `Login Data` extraction described in Class C

— months or years before. Ticketmaster (560M records), AT&T (109M call logs), Santander, Advance Auto Parts: all downstream from "someone's browser had saved passwords." The earliest infostealer infection date Mandiant traced back to **November 2020** — a four-year fuse.

The Snowflake parallel is the slide that lands the meeting: if NSU's lab machines today look like the contractor laptops that fed the Snowflake breach, then NSU is one infostealer infection — or one curious student with NirSoft ChromePass — away from a comparable internal incident. *Without anyone phishing anyone.*

In 2024 alone, infostealers infected **4.3 million machines** and produced **330 million credential records**. The credentials-list market has **3.9 billion** credentials sourced from infostealer logs. RedLine alone accounted for **34%** of global infostealer infections in 2024. The Verizon 2025 DBIR shows **over half of ransomware victims** had domains in stealer logs before the ransomware attack. Hudson Rock tracks **30+ million** infected computers across the infostealer ecosystem.

The architectural fix has been documented for a decade. Almost no university deploys it. **That gap is your paper.**

3. THE FIX — by attack class, free / near-free

3.1 — Kill Class A (Forgotten Profile) and Class B (Sync Bleed)

Three Chrome Enterprise policies. Deploy via Group Policy (free, NSU presumably has AD already) or **Chrome Browser Cloud Management** (free for unlimited browsers, sign in with any Google account, gets you a web console to push policy to every managed Chrome).

Registry path (Windows, can be pushed via GPO or .reg file):

`HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Google\Chrome`

Policy name	Value	What it does
BrowserSignin	0 (DWORD)	Disables browser sign-in entirely. Students can still use Gmail in a tab — they just can't sign Chrome itself into a Google profile. Eliminates Class A and B at the root.
SyncDisabled	1 (DWORD)	Belt-and-braces — even if sign-in is allowed somewhere, sync is forbidden.
RestrictSigninToPattern	".*@northsouth\\.edu" (STRING)	If you keep sign-in enabled for some lab roles, only allow @northsouth.edu accounts. Prevents personal accounts entirely.
PasswordManagerEnabled	0 (DWORD)	Disables the built-in Chrome password manager. Nothing gets saved to Login Data.
AutofillAddressEnabled	0 (DWORD)	Disables address autofill.
AutofillCreditCardEnabled	0 (DWORD)	Disables payment-method autofill.
BrowserGuestModeEnforced	1 (DWORD)	Forces Guest Mode — no profile data persists between sessions.
DeveloperToolsAvailability	2 (DWORD)	Disables DevTools — prevents students from extracting cookies/storage from F12 console.
ExtensionInstallBlocklist	["*"] (STRING list)	Block all extensions by default.
ExtensionInstallAllowlist	["id1", "id2", ...]	Only approved extensions (e.g., a chosen ad-blocker, the NSU library proxy if any).
BlockExternalExtensions	1 (DWORD)	Blocks side-loaded extensions from external sources.

Equivalent for Microsoft Edge: same policy names live under

`HKLM\SOFTWARE\Policies\Microsoft\Edge`. Edge honors `BrowserSignin`, `SyncDisabled`, `PasswordManagerEnabled`, `AutofillCreditCardEnabled`, etc. Push both — students will install Firefox if you only lock Chrome, so plan for Firefox policy via `policies.json` in `C:\Program Files\Mozilla Firefox\distribution\` too.

Deployment path for NSU (free):

1. **Chrome Browser Cloud Management** (chromeenterprise.google) — sign up with the NSU admin Google account, enroll lab machines with an enrollment token. **Zero cost. No license. Unlimited browsers.** Push all the above policies from a web UI.
2. **Group Policy** — download Chrome ADMX templates from chromeenterprise.google, drop into `\\domain\SYSTEM\domain\Policies\PolicyDefinitions`, push via standard GPO. Free, uses existing AD.
3. **Standalone PCs (no domain):** generate a `.reg` file, run once at imaging time.

3.2 — Kill profile persistence: Windows Shared PC Mode

Built into Windows 10/11. Zero cost. Almost nobody uses it.

Shared PC mode (configured via the SharedPC CSP, or in Group Policy under Computer Configuration → Administrative Templates → Windows Components → Shared PC) does this automatically:

- Deletes user profiles **immediately on sign-out** (or on disk threshold, or on inactivity threshold — your choice).
- Disables creating new local accounts.
- Disables sleep mode and forces sign-out after inactivity.
- Removes the "remember me" options on the login screen.

The relevant policies (deploy via Intune Custom OMA-URI, local GPO, or `SharedPC` CSP):

Setting	Recommended value
EnableSharedPCMode	True
AccountModel	Domain-joined only (or Guest only) for fully kiosk-style labs)
DeletionPolicy	0 = delete immediately at sign-out
SignInOnResume	True
SleepTimeout	0 (no sleep on shared device)
MaintenanceStartTime	nightly window
Plus GPO "Delete user profiles older than a specified number of days on system restart" = 1 day, as a safety net	

When this is on, even if a student leaves Chrome signed in with sync enabled, **the entire profile folder is wiped on logoff**. Class A and B die. Class C has nothing to extract.

Known gotchas:

- Locally created accounts that exist *before* enabling Shared PC mode are not auto-deleted; you need to clean those up first.
- On Windows 11 Pro vs Education edition, some behaviors differ. Test on one machine first.

3.3 — Kill Class D (Malicious Extensions)

- ExtensionInstallBlocklist = ["*"] + a tiny allowlist. Done.
- Add BlockExternalExtensions = 1 to stop side-loads.
- For staff laptops (not lab PCs) where you can't be as draconian: deploy **Password Alert** (Google's free open-source extension, github.com/google/password-alert) as a forced install via ExtensionInstallForcelist — it alerts when a student types their @northsouth.edu password into any non-Google site. Combined with the Google Safe Browsing built into Chrome, this catches credential reuse on phishing pages.

3.4 — Kill Class E (Native Hidden Keylogger)

This is where most universities fail. The fix is layered, all free:

Layer 1: Remove local admin from lab student accounts. Standard user only. Use a separate lab-admin account for IT.

Layer 2: Microsoft Defender ASR (Attack Surface Reduction) rules. All free, already on every Windows 10/11 machine with Defender enabled. Enable via PowerShell (one-liner per rule) or Intune/GPO. High-value rules for keylogger/credential-theft defense:

Rule GUID	What it blocks
<code>9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2</code>	Block credential stealing from LSASS (Mimikatz, ProcDump-style memory dumps). Microsoft made this Block by default in 2024 , but verify it's actually on. This rule does not support Warn mode — use Audit or Block only.
<code>d4f940ab-401b-4efc-aadc-ad5f3c50688a</code>	Block all Office applications from creating child processes
<code>3b576869-a4ec-4529-8536-b80a7769e899</code>	Block Office applications from creating executable content
<code>d3e037e1-3eb8-44c8-a917-57927947596d</code>	Block JavaScript or VBScript from launching downloaded executable content
<code>5beb7efe-fd9a-4556-801d-275e5ffc04cc</code>	Block execution of potentially obfuscated scripts
<code>7674ba52-37eb-4a4f-a9a1-f0f9a1619a2c</code>	Block Adobe Reader from creating child processes
<code>26190899-1602-49e8-8b27-eb1d0a1ce869</code>	Block Office communication app (Outlook) from creating child processes
<code>e6db77e5-3df2-4cf1-b95a-636979351e5b</code>	Block persistence through WMI event subscription
<code>b2b3f03d-6a65-4f7b-a9c7-1c7ef74a9ba4</code>	Block untrusted USB processes (USB threats)
<code>01443614-cd74-433a-b99e-2ecdc07bfc25</code>	Block executable files from running unless they meet a prevalence, age, or trusted list criterion

PowerShell example:

```
powershell
```

```
Set-MpPreference -AttackSurfaceReductionRules_Ids `
9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2 `
-AttackSurfaceReductionRules_Actions Enabled
```

Set all of the above to **Enabled** (Block mode) on lab machines **after a week in AuditMode** to find false positives (Patch My PC and a few other legit tools sometimes trigger the LSASS rule and need exclusions).

Layer 3: PowerShell Constrained Language Mode on student accounts. Set environment variable `__PSLockdownPolicy = 4` system-wide or via AppLocker policy → PowerShell drops into Constrained Language Mode → most malicious PowerShell scripts (which rely on .NET reflection) break.

Layer 4: AppLocker (free on Windows Pro/Enterprise/Education) — whitelist allowed executables. At minimum, deny:

- `C:\Users\*\AppData\` (block executables from user-writable AppData)
- `C:\Users\*\Downloads\`
- Block `pythonw.exe`, `wscript.exe`, `cscript.exe`, `mshta.exe` from standard users, or allow only from `C:\Program Files\`.

Layer 5: Block USB autorun and unknown USB devices. GPO: Computer Configuration → Administrative Templates → System → Removable Storage Access → "All Removable Storage classes: Deny all access" for student accounts, or use Defender's device control. (Hardware keylogger defense is the harder problem — see § 3.5.)

Layer 6: Sysmon + Wazuh for detection. Specific rules to add for this attack surface:

- **Sysmon Event ID 11 (FileCreate)** alert on file paths matching `*\Login Data` written to anywhere *outside* `*\Google\Chrome\User Data\*` (catches a tool copying the SQLite DB to extract elsewhere).
- **Sysmon Event ID 10 (ProcessAccess)** alert on any process opening `lsass.exe` with `GrantedAccess` matching `0x1010` or `0x1410` (LSASS memory read — Mimikatz / LaZagne behavior). Wazuh has out-of-the-box rule families for this; the Wazuh blog "Snake keylogger detection with Wazuh" and "Detecting process injection attacks with Wazuh" both give the exact rule XML you can copy.
- **Sysmon Event ID 1 (ProcessCreate)** alert on `pythonw.exe`, `wscript.exe`, `mshta.exe`, `powershell.exe -WindowStyle Hidden` spawned from user contexts.
- **Sysmon Event ID 12/13 (RegistryEvent)** alert on writes to `HKCU\Software\Microsoft\Windows\CurrentVersion\Run` and `HKLM\...\Run` — classic keylogger persistence.

- **Sysmon Event ID 3 (NetworkConnect)** alert on outbound connections to `api.telegram.org`, `discord.com/api/webhooks`, `pastebin.com`, `transfer.sh`, and known anonymous-uploaders from any non-browser process.
- **Sysmon Event ID 25 (ProcessTampering)** — process hollowing / image replacement.

Wazuh active-response can automatically kill the process and quarantine the binary on these alerts. The whole stack — Sysmon + Wazuh + custom rules — is **free**.

3.5 — Hardware Keyloggers (Class F)

Mostly procedural:

- Tamper-evident stickers on USB ports (cheap, BDT 10/sticker).
- Lab attendant inspection at start and end of each day.
- Use PS/2 keyboards on the highest-risk machines (registrar, finance) — PS/2 inline keyloggers exist but are rarer and more obvious.
- Long-term: USB port blocking via Group Policy for non-allowlisted device IDs (Defender Device Control or third-party).

3.6 — Don't recommend Chrome's built-in password manager. Recommend Bitwarden.

For staff (not lab PCs):

- **Bitwarden** is free, open source, end-to-end encrypted, **master-password locked** (you can't view passwords with just the OS user password), auto-locks on browser close, requires re-auth to export. It's not vulnerable to the Class A or Class C attacks on Chrome's password store.
- **Bitwarden Teams Starter** is **free for 10 users**, then ~USD 4/user/month — affordable for IT, registrar, finance, deans. Or self-host **Vaultwarden** (open source, Docker container, supports unlimited users) for zero cost.
- Policy: disable Chrome/Edge built-in password manager (`PasswordManagerEnabled = 0` above), recommend Bitwarden via the official extension, push as `ExtensionInstallForcelist` for managed machines.

4. THE LIVE DEMO (the bomb)

This is what wins the meeting. Set up two scenarios on lab PCs the IT director controls. Use **your own throwaway Google account** — never demo with real student data.

Scenario 1 — "30 seconds, no tools"

1. On a current NSU lab PC, sign into Chrome with `throwaway@gmail.com`, save 5-6 fake

passwords (`facebook.com`), (`bkash.com`), etc., with dummy creds you control).

2. Close Chrome. Walk away. Pretend you're the next student.
3. Sit back down. Open Chrome. Click the avatar — see (`throwaway@gmail.com`) still there.
4. Open (`chrome://settings/passwords`). Click the eye on any entry. Type the Windows password (whatever the shared lab password is). **All passwords visible. Click the three-dot menu → Export passwords → CSV.**
5. Hand the CSV to the director.

Total time: under one minute. No tools. No "hacking." Default behavior.

Scenario 2 — "5 minutes, free tool"

1. On a freshly imaged identical lab PC (sync was on, then user signed out), drop ChromePass.exe (downloaded from NirSoft, ~150 KB) onto the desktop from a USB.
2. Double-click. Window opens showing every saved password from the prior session **even though Chrome is signed out** — because the local profile data persists.
3. Export → CSV. Same outcome.

Scenario 3 — "Now watch the fix" (the close)

1. Show the same lab PC after deploying (`BrowserSignin=0`), (`SyncDisabled=1`), (`PasswordManagerEnabled=0`), Shared PC mode immediate-delete-on-logout.
2. Sign in attempt: Chrome refuses. "Sync is disabled by your administrator."
3. Try to save a password: option is greyed out.
4. Sign out and sign back in: profile folder is empty. No (`Login Data`) exists.
5. Run ChromePass: nothing to extract. Empty output.

This is the bomb. Don't oversell. Let the silence do the work.

Ethics note: do all three scenarios with the IT director or a faculty supervisor present, using only accounts you own, on machines they've authorized. Anything else risks crossing into the Bangladesh Cyber Security Ordinance 2025's unauthorized-access clauses.

5. RESEARCH PAPER — making this publishable

Working title options

- *"The Forgotten Attack Surface: Browser Profile Persistence and Sync Leakage on Shared University Endpoints"*
- *"Sync, Forgotten: A Measurement Study of Default Browser Behavior on Shared Lab Computers in a South Asian University"*

- *"Don't Put the Juice on the Table: An Architectural Approach to End-User Credential Protection in a Bangladeshi University"*

Why it's publishable

- **Genuine gap in the literature.** Browser security research focuses on phishing, password manager cryptography, or zero-day vulnerabilities. The *deployment*-level question of "what does a default Chrome install do on a shared university PC, and what is leaked between users?" has remarkably little measurement-study coverage, especially in South Asian universities.
- **Bangladesh / South Asian university data is nearly absent** from venues like SOUPS, USENIX Security, ACSAC, IEEE S&P. A measurement + intervention paper from a 25,000-user Bangladeshi university is novel ground.
- **It connects directly to the Snowflake-class breach pattern** that is the dominant credential-theft story of 2024–2026. The literature is screaming about infostealers; nobody has shown what the analogue looks like on shared university endpoints, measured before and after architectural intervention.

Target venues (in fit order)

1. **SOUPS (Symposium on Usable Privacy and Security)** — fits perfectly; open-access proceedings on usenix.org; co-located with USENIX Security.
2. **USENIX Enigma** — short, practice-oriented; deployments welcome.
3. **ACSAC Case Studies track** — built for real-world deployment papers.
4. **Journal of Cybersecurity (Oxford)** — extended journal version.
5. **IEEE Access** — fast turnaround, open access, lower bar; useful as the first paper to break the ice.
6. **NDSS, CCS, IEEE S&P** — only if you go very deep on the measurement and the intervention is rigorously controlled. Long shot but not impossible if the data is strong.

Study design

Measurement phase (no intervention yet — 4–6 weeks)

1. **Audit policy state.** Enumerate every NSU lab PC. Run a script (PowerShell + `gpresult /h`) on each that records: is `SyncDisabled` set? Is `BrowserSignin` set? Is `PasswordManagerEnabled` disabled? Is Shared PC mode on? Is Defender ASR LSASS rule on? Are extensions whitelisted? Catalogue current state, anonymized per lab.
2. **Default behavior measurement.** On a representative sample of lab PCs, with IRB approval and **no user identification**, measure: (a) does Chrome offer "turn on sync" prompts by default? (b) does the local profile persist after sign-out? (c) is `Login Data` writable by a standard user? (d) does a standard-user-context tool (LaZagne, against a seeded test

account only) successfully read the database? Report at the lab level, not user level. No actual student credentials are read or stored.

3. **Indirect prevalence measurement** (ethically delicate; needs careful IRB framing). With explicit opt-in consent from a sample of student volunteers, run a script that simply *counts* the number of entries in `Login Data` (without reading them) and reports the count to the researcher. Tells you "how many passwords does the average lab profile have at any moment" without ever touching a real credential.
4. **Anonymous self-report survey** (Bangla + English) — N target 500 — on student behavior: do you sign into Chrome on lab PCs? Do you turn on sync? Do you sign out? Have you ever found a previous user's profile still signed in? Have you ever saved a password to a lab PC?

Intervention phase (8-12 weeks)

1. Deploy the architectural stack from § 3 on Lab A. Leave Lab B as control. Lab A and B should be matched in size, course usage, and student population.
2. Re-measure the policy-state audit (verify deployment).
3. Run a *simulated attacker* assessment on Lab A vs Lab B: standard-user-context attempts to extract saved credentials post-session, **using only seeded test accounts you control**. Pass/fail per machine. Time to extract.
4. Re-run the survey.

Post-intervention phase (4 weeks)

1. Look for unintended consequences: did students compensate by installing Firefox? By using personal phones for everything? By creating shared password files in Google Drive? Survey + interview a sub-sample.

Statistics

- Lab-level paired comparison: pre vs post on the same Lab A; cross-sectional comparison Lab A vs Lab B post-intervention.
- Primary outcome: simulated-extraction success rate per machine, pre vs post.
- Secondary outcomes: number of `Login Data` entries (with opt-in consent), self-reported sync usage, self-reported "previous user still signed in" incidents.
- Mixed-effects logistic regression with lab as random effect.

Ethics — non-negotiable rules

- At no point does the research team read, store, or transmit any actual student credential. Period. Even in encrypted form. Even briefly.
- The "extraction success" measurement uses *seeded* test accounts the research team owns, not real student accounts.

- The `Login Data` entry-count measurement is opt-in and reports only an integer.
- All deployment work happens with IT department co-supervision, not unilaterally.
- The lab-level policy audit is announced to lab users (signage) so it's not covert.
- Pre-register the protocol on OSF before any data collection. Pre-registration is *the* signal of methodological seriousness for SOUPS reviewers.

Related work to cite

- Lain, Kostiainen, Čapkun (IEEE S&P 2022) — phishing training large-scale field study.
- Lain et al. (ACM CCS 2024) — embedded training and nudges follow-up.
- Steves, Greene, Theofanos (NIST, *Journal of Cybersecurity* 2020) — Phish Scale.
- Mandiant (2024) "UNC5537 and the Snowflake campaign" — infostealer-to-breach pipeline.
- IBM X-Force 2025 Threat Intelligence Index — Lumma / RisePro / Vidar / Stealc / RedLine prevalence.
- Verizon 2025 DBIR — over half of ransomware victims had prior stealer-log exposure.
- Elastic Security Labs (Oct 2024) "Katz and Mouse Game" — Chrome 127 App-Bound Encryption and the bypass arms race.
- Carnegie Mellon ISO (2025) advisory on the December 2024 Chrome extension supply chain wave.
- arXiv 2503.04292 (2025) "A Study on Malicious Browser Extensions in 2025."
- Hudson Rock infostealer ecosystem research (2024–2026).
- Microsoft Learn — SharedPC CSP / Shared PC mode documentation.
- Microsoft Defender ASR rules reference (`defender-docs`) on GitHub).
- Bangladesh Cyber Security Ordinance 2025 and Personal Data Protection Ordinance 2025.

The novelty paragraph reviewers want to see

"While the literature on phishing training, password manager cryptography, and infostealer malware ecosystems is now substantial, the deployment-level question — what does the default Chrome configuration leak between users on a shared university endpoint, and what is the effect of low-cost architectural intervention — has received remarkably little measurement attention, particularly outside North American and European institutional contexts. We close this gap with a controlled measurement-and-intervention study at a 25,000-user Bangladeshi university, demonstrating that three free Chrome Enterprise policies and one built-in Windows feature reduce simulated-attacker credential extraction success from X% to Y% across N matched lab machines, while making the case that the architectural fix is universally deployable at zero license cost."

6. PITCH STRATEGY (keeping the project yours)

This is the part you were worried about — "they take my idea and ditch me." Counter-moves:

1. **Bring the demo, not just the slides.** A live demo on a real NSU lab PC is not something a senior administrator can take and run themselves next week. You become the person who *can* do it. Schedule the demo for meeting 1, not as a follow-up.
 2. **Don't hand over the full technical document on the first meeting.** Show them the one-page summary (§ 0) and the demo. Hand over the rest only after a written agreement is in place.
 3. **Frame as a formal supervised research project from the start.** Bring a one-page MoU draft to meeting 2. Required clauses:
 - Sajid as PI / first author on any resulting publication.
 - Faculty supervisor named (Dr. Shifat-E-Rabbi or another willing CSE faculty).
 - IT Director as project sponsor and co-author on the deployment paper.
 - Defined deliverables: pilot deployment on Lab A, measurement report, intervention report, draft paper.
 - Defined timeline: 9 months.
 - Stipend or formal internship letter — even unpaid, the letter is essential for the Australian Master's apps.
 4. **Open source the work from day 1.** Push the deployment scripts (PowerShell, GPO XMLs, Wazuh rules) to your personal GitHub under MIT or Apache 2.0, credit NSU clearly. *Your name is on the repo. The repo predates any institutional claim.* This is the strongest possible portfolio anchor for QUT / Deakin / RMIT / Griffith / ECU admissions.
 5. **Stage-gate deliverables.** Each phase ends in a written, dated, countersigned report. The paper trail is bulletproof if there's ever a dispute about contribution.
 6. **The career angle is real.** A SOUPS paper as first author from a Bangladeshi undergraduate is a *standout* signal for Australian Master's admissions in cybersecurity. The committee is reading hundreds of applications from candidates with strong GPAs and generic projects; almost none of them will have a deployed, measured, published security intervention. This is the kind of single artifact that overrides CGPA concerns.
-

7. THE 30-DAY PERMISSION ASK (what you actually request in meeting 1)

You're not asking for budget. You're not asking for headcount. You're asking for **one lab** for **one month** to demonstrate the intervention works. Specifically:

- One lab of ~30 machines, with IT department co-supervision.
- Authorization to:

1. Run the policy-state audit script (read-only, no credentials touched).
 2. Deploy the Chrome Enterprise policies via GPO or Chrome Browser Cloud Management.
 3. Enable Windows Shared PC mode (immediate deletion on sign-out).
 4. Enable the named Defender ASR rules (audit first, block second).
 5. Install Sysmon + Wazuh agent in audit mode.
- One co-author letter from IT director and one faculty supervisor.
 - Permission to use anonymized aggregate data for a research paper, subject to IRB approval.

Cost to NSU: zero rupees, zero taka. All software is already licensed (Windows, Defender) or free (Chrome policies, Wazuh, Sysmon). Hardware: existing lab machines.

Risk: minimal. If a deployed policy breaks a workflow, the policy is reversible in minutes via GPO. Wazuh runs in audit mode — it observes only, doesn't block.

Reward: a measurable, publishable, demonstrable reduction in NSU's most-overlooked attack surface, plus a paper that puts NSU's name on the international cybersecurity research map.

8. CAVEATS (be honest — it strengthens the pitch)

- **NSU's actual current state is unmeasured.** Some of these policies *might* already be deployed and just not visible to you. The audit script will tell you in week 1. If 80% of the controls are already on, the paper becomes about the remaining 20% and about behavioral measurement — still publishable.
- **Chrome's App-Bound Encryption (introduced July 2024 in Chrome 127)** complicates *cookie* extraction by tying decryption to the Chrome process identity. Saved *passwords* are still DPAPI-encrypted as before — the App-Bound change applies primarily to cookies. The Class C attack on passwords still works in 2026. Note this carefully in the paper.
- **Modern infostealers actively bypass App-Bound Encryption** — Elastic Security Labs documented the Lumma/MetaStealer/XenoStealer bypass within weeks of the Chrome 127 release. The arms race is real.
- **The Tackker-class browser extension** behaves like a keylogger once installed. Forced extension allowlisting prevents installation; it does *not* remove already-installed malicious extensions on personal devices. Honest paper section.
- **Shared PC mode** changes the user experience. Students lose the ability to "leave my session open and come back." Communicate the trade-off; it's worth it for shared labs but be transparent.
- **Defender ASR LSASS rule** can occasionally interfere with legitimate software (e.g., some patch managers). Deploy in audit mode for 1–2 weeks, review exceptions, then move to

block. Document everything.

- **The custom browser extension from the previous report — drop it for this pitch.** You said so yourself. The angle here is *removing the attack surface*, not adding more code. The extension can be a Phase 2 idea.
 - **Hardware keyloggers are real and software cannot defeat them.** Be clear in the pitch that physical inspection + tamper-evident stickers are the only defense.
 - **None of this is a substitute for what was in the previous policy report:** DMARC, 2SV/passkeys on staff, Gmail anti-phishing, phishing simulations. That stack and this stack are *complementary*. The previous report defended the inbox; this report defends the endpoint.
 - **All demos must be on machines you have explicit, written authorization to test on, using only accounts you own.** The Cyber Security Ordinance 2025 has unauthorized-access clauses with real penalties. Get the authorization in writing before scenario 1.
-

9. THE FINAL LINE

The previous report told NSU how to stop attackers from putting juicy food on the table. This report tells NSU how to take the food off the table that's already there.

Every lab PC at NSU today is a credential aggregator. Every Chrome profile that gets signed in with sync turned on is downloading a personal life onto a public computer. Every session that ends without a wipe leaves the next student a key to the previous student's accounts. No phishing. No exploit. Just default behavior, working as designed, on machines NSU owns.

The fix is free, documented, supported by Microsoft and Google themselves, and almost nobody deploys it.

That's the bomb. That's the paper. That's the meeting.

Sajid — open the meeting with the live demo. Show, don't tell. Hand them the one-pager (§ 0). Walk out with the 30-day pilot.